HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

PROJECT 01

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

HỆ ĐA AGENT HỖ TRỢ ĐÁNH GIÁ NGUY CƠ BỆNH TIỂU ĐƯỜNG

Single Agent và Multi Agent trên cùng mô hình, dữ liệu và kho tri thức

Giảng viên hướng dẫn	PGS.TS Trần Đình Quế
Sinh viên thực hiện	Nguyễn Minh Vinh
Mã sinh viên	B23DCCN934
Lớp	D23CTPM01
Nhóm	08

Hà Nội, tháng 11 năm 2026

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT

DANH MỤC HÌNH VÀ BẢNG

TÓM TẮT

CHƯƠNG 1 GIỚI THIỆU

1.1 Bài toán và động lực

1.2 Mục tiêu và phạm vi

1.3 Đóng góp và cấu trúc báo cáo

CHƯƠNG 2 HỆ ĐA AGENT

2.1 Agent

2.2 Hệ đa agent

2.3 Giao tiếp và trạng thái dùng chung

2.4 Guardrail và khả năng kiểm toán

CHƯƠNG 3 KIẾN TRÚC TỔNG QUAN

3.1 Ánh xạ đề bài sang hệ thống

3.2 Vai trò của các agent

3.3 Luồng một lượt xử lý

3.4 Công nghệ và tổ chức mã nguồn

CHƯƠNG 4 CÁC THÀNH PHẦN CỦA HỆ THỐNG

4.1 Machine Learning thành Prediction Agent

4.1.1 Dữ liệu và bài toán

4.1.2 Tiền xử lý và chia dữ liệu

4.1.3 Mô hình và huấn luyện

4.1.4 Từ mô hình thành agent

4.2 Knowledge Base và RAG

4.2.1 Nguồn tri thức

4.2.2 BM25 và Knowledge Graph

4.2.3 Kiểm soát bằng chứng

4.3 Statistics Agent

4.4 Validation và Safety

4.5 Single Agent và Multi Agent

4.6 PyTorch Neo4j và điều phối có phản biện

CHƯƠNG 5 CODE, DEMO VÀ ĐÁNH GIÁ

5.1 Cấu trúc mã nguồn

5.2 Các lớp và hàm chính

5.3 Cách chạy

5.4 Demo câu hỏi kiến thức

5.5 Demo đánh giá hồ sơ

5.6 Demo safety và ngoài phạm vi

5.7 Thiết kế benchmark

5.8 Kết quả và phân tích

5.9 Triển khai Render

5.10 Hạn chế và hướng phát triển

5.11 Notebook thực thi và giải thích từng cell

5.12 Code huấn luyện và kết quả PyTorch

KẾT LUẬN

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Ý nghĩa trong project
API	Application Programming Interface	Giao diện dịch vụ FastAPI
AUC	Area Under the Curve	Thước đo xếp hạng mô hình
BM25	Best Matching 25	Thuật toán truy xuất văn bản
BMI	Body Mass Index	Chỉ số khối cơ thể
CDC	Centers for Disease Control and Prevention	Nguồn bộ dữ liệu thống kê
F1	F1-score	Trung bình điều hòa precision và recall
KG	Knowledge Graph	Đồ thị tri thức có quan hệ và nguồn
MAS	Multi-Agent System	Hệ nhiều tác tử chuyên trách
MLP	Multilayer Perceptron	Mô hình được chọn cho risk score
RAG	Retrieval-Augmented Generation	Trả lời dựa trên tài liệu truy xuất
ROC	Receiver Operating Characteristic	Đường cong đánh giá phân loại

DANH MỤC HÌNH VÀ BẢNG

Danh mục hình

Hình 3.1 Kiến trúc tổng quan của DiaAgents

Hình 3.2 Luồng xử lý một yêu cầu

Hình 3.3 Thành phần triển khai và nơi lưu dữ liệu

Hình 4.1 So sánh metric của ba mô hình trên tập test

Hình 5.1 Benchmark Single Agent và Multi Agent

Danh mục bảng

Bảng 1.1 Mục tiêu và artifact kiểm chứng

Bảng 2.1 Single Agent và Multi Agent

Bảng 2.2 Trạng thái đầu ra

Bảng 3.1 Vai trò của các agent

Bảng 3.2 Công nghệ sử dụng

Bảng 4.1 Thống kê dataset dự đoán

Bảng 4.2 Chia train-validation-test

Bảng 4.3 Kết quả mô hình

Bảng 4.4 Corpus và Knowledge Graph

Bảng 5.1 Cấu trúc mã nguồn

Bảng 5.2 Endpoint API

Bảng 5.3 Kết quả benchmark

Bảng 5.4 Nhóm tình huống benchmark

TÓM TẮT

Project xây dựng DiaAgents, một hệ hỗ trợ học tập về nguy cơ bệnh tiểu đường, trong đó cùng một yêu cầu được xử lý bởi baseline Single Agent hoặc hệ Multi-Agent. Hai kiến trúc dùng chung checkpoint, dataset, kho tri thức, thuật toán thống kê và policy; nhờ vậy phép so sánh tập trung vào cách tổ chức và khả năng quan sát thay vì lợi thế dữ liệu hoặc mô hình.

Khối dự đoán gồm Logistic Regression, Random Forest, MLP sklearn và MLP PyTorch trên Diabetes Prediction Dataset 100.000 dòng. PyTorch của demo đạt test F1 0.7924, dùng threshold 0.390; model sklearn được giữ để đối chiếu. CDC BRFSS 2015 có 253.680 dòng dùng thống kê mô tả. Neo4j lưu tám tài liệu và 16 quan hệ tri thức. MessageBus, Critic và Q-learning phối hợp các bước xử lý có kiểm soát.

Benchmark kiểm soát gồm 58 tình huống cho mỗi kiến trúc. Hai hệ cùng đạt 100% routing accuracy, task success, citation recall và safety block trên bộ contract hiện tại. Multi-Agent tạo trace chuyên trách cho toàn bộ case nên audit coverage đạt 100%, trong khi Single Agent chỉ có một trace tổng hợp. Kết quả này minh họa lợi ích kiểm toán của việc tách tác tử; không được diễn giải thành bằng chứng rằng Multi-Agent luôn chính xác hơn hoặc hệ thống có giá trị chẩn đoán.

CHƯƠNG 1 GIỚI THIỆU

1.1 Bài toán và động lực

Một mô hình dự đoán đơn lẻ có thể tạo risk score từ hồ sơ, nhưng chưa giải quyết toàn bộ nhu cầu của một hệ thống thông minh: kiểm tra đầu vào, giải thích bằng nguồn, trả lời câu hỏi kiến thức, tính thống kê, từ chối yêu cầu nguy hiểm và ghi lại bước nào đã tạo kết quả. Khi những trách nhiệm này nằm trong một hàm lớn, lỗi khó định vị và người dùng khó biết kết luận đến từ model, dữ liệu hay luật.

Bệnh tiểu đường là miền phù hợp để minh họa kiến trúc vì dữ liệu có cả số, danh mục và tri thức văn bản. Tuy nhiên đây cũng là miền nhạy cảm: một score học thuật không thể thay thế xét nghiệm hoặc chẩn đoán. Do đó project đặt validation, citation và safety thành thành phần kiến trúc thay vì chỉ ghi chú ở cuối giao diện.

Ý tưởng trung tâm là tách các trách nhiệm thành agent chuyên trách nhưng giữ chung dịch vụ nền. Coordinator định tuyến, Validation kiểm tra schema, Prediction gọi checkpoint, Statistics tính trực tiếp từ CDC, Retrieval và Graph tìm bằng chứng, Explanation tổng hợp, còn Verifier Safety quyết định trạng thái cuối. Baseline Single Agent dùng cùng dịch vụ nhưng thực hiện trong một khối để tạo đối chứng công bằng.

1.2 Mục tiêu và phạm vi

Mục tiêu thứ nhất là tạo một pipeline dự đoán có thể tái lập: dữ liệu được kiểm tra, chia train-validation-test có phân tầng, threshold và model được chọn trên validation, test chỉ dùng báo cáo cuối. Mục tiêu thứ hai là nối model với RAG, Knowledge Graph và thống kê chính xác trong một API thống nhất.

Mục tiêu thứ ba là so sánh Single Agent và Multi-Agent trên cùng đầu vào và cùng công cụ. Bộ benchmark phải đo đúng định tuyến, hoàn thành nhiệm vụ, citation, guardrail và trace thay vì chỉ quan sát một vài câu trả lời đẹp. Mục tiêu thứ tư là có notebook đã chạy và web có thể triển khai bằng Render Blueprint.

Phạm vi của project là sàng lọc học thuật và minh họa kiến trúc. Hệ thống không nhận ảnh y khoa, không phân tích đơn thuốc, không kê liều, không lưu hồ sơ định danh và không thay thế nhân viên y tế. Corpus nhỏ nên câu hỏi ngoài phạm vi hoặc thiếu bằng chứng phải được trả trạng thái rõ ràng thay vì tạo câu trả lời khẳng định.

Mục tiêu	Cách thực hiện	Artifact kiểm chứng
Dự đoán tái lập	Pipeline + checkpoint + metadata	artifacts/model_metadata.json
Tri thức có nguồn	BM25 + KG + citation guard	knowledge/corpus.json, graph.json
So sánh công bằng	Cùng Services và cùng 58 case	results/agent_benchmark.csv

Mục tiêu	Cách thực hiện	Artifact kiểm chứng
Demo chạy được	FastAPI + notebook + Render	app, notebooks, render.yaml

1.3 Đóng góp và cấu trúc báo cáo

Đóng góp chính của project là một hệ thống đầu-cuối gồm model, hai dataset, kho tri thức, đồ thị, API, giao diện, notebook và benchmark. Mọi số liệu trong báo cáo được đọc từ artifact của repository; báo cáo không gán cho hệ thống khả năng điều trị hay hiệu quả lâm sàng chưa được kiểm chứng.

Chương 2 trình bày agent, hệ đa agent, giao tiếp bằng State và trace. Chương 3 ánh xạ yêu cầu sang kiến trúc, mô tả vai trò và công nghệ. Chương 4 đi vào model, dữ liệu, RAG, KG, statistics, validation và safety. Chương 5 trình bày code, demo, benchmark, triển khai và giới hạn.

CHƯƠNG 2 HỆ ĐA AGENT

2.1 Agent

Agent là thành phần nhận quan sát, duy trì trạng thái cần thiết, chọn hành động và trả kết quả hướng tới mục tiêu. Trong project, quan sát là câu hỏi và hồ sơ; trạng thái là State có schema; hành động là gọi model, truy xuất, tính thống kê, chặn yêu cầu hoặc tạo câu trả lời có nguồn.

Một agent không nhất thiết phải dùng mô hình ngôn ngữ. Coordinator của project dùng rule định tuyến có thể kiểm tra; Statistics Agent dùng phép tính DataFrame; Prediction Agent dùng checkpoint; Retrieval Agent dùng BM25; Verifier dùng contract bằng chứng và safety. Cách tiếp cận lai giúp kết quả ổn định và chạy cục bộ.

Chu trình xử lý gồm nhận input, chuẩn hóa, chọn intent, gọi công cụ, cập nhật State, kiểm tra điều kiện và phát đầu ra. Nếu validation thất bại hoặc yêu cầu nguy hiểm, chu trình kết thúc sớm trước model. Đây là hành vi chủ ý để giảm bề mặt rủi ro.

2.2 Hệ đa agent

Hệ đa agent phân rã một nhiệm vụ thành nhiều vai trò. Lợi ích là mỗi vai trò có contract nhỏ, dễ thay thế và dễ quan sát. Chi phí là cần coordinator, schema dùng chung và quy tắc kết thúc; nếu phân rã không hợp lý, nhiều agent chỉ làm tăng độ phức tạp mà không tăng chất lượng.

Project không so một Multi-Agent mạnh với một Single Agent yếu. Cả hai nhận cùng question và profile, dùng cùng DiabetesModelService và KnowledgeService. Single Agent vẫn có validation, prediction, retrieval và safety nhưng chỉ ghi một trace tổng hợp. Multi-Agent ghi trace theo specialist, nhờ đó có thể biết bước nào chạy và bước nào dừng.

Tiêu chí	Single Agent	Multi Agent
Tổ chức	Một khối xử lý toàn bộ	Nhiều tác tử chuyên trách
Model và dữ liệu	Dùng chung	Dùng chung
Trace	Một bước tổng hợp	Nhiều bước có tên agent
Định vị lỗi	Khó xác định công đoạn	Có thể xác định specialist
Chi phí phối hợp	Thấp	Có thêm điều phối và State
Độ chính xác mặc định	Không đảm bảo thấp hơn	Không đảm bảo cao hơn

2.3 Giao tiếp và trạng thái dùng chung

Các agent trao đổi qua đối tượng State thay vì chuỗi văn bản tự do. State gồm mode, question, profile, request_id, intent, prediction, statistics, knowledge, validation_errors, traces, status, answer và safety_flags. Schema cố định giúp notebook, API và giao diện đọc cùng cấu trúc.

Trace ghi agent, status, duration_ms, summary và tool. total_duration_ms đo wall time từ đầu đến serialize. MessageBus lưu REQUEST, INFORM, CHALLENGE, RESPONSE hoặc FAILURE. Message có thể chứa profile nên demo không nhập dữ liệu định danh. Bus chạy trong tiến trình, chưa có broker phân tán hoặc persistence.

Giao tiếp được tổ chức theo pipeline có điều kiện. Coordinator chọn intent; Validation có thể chặn; specialist ghi kết quả vào State; Explanation chỉ đọc kết quả đã tạo; Verifier kiểm tra citation và cờ an toàn trước khi trả trạng thái cuối. Nhờ vậy mỗi agent có đầu vào và đầu ra rõ ràng.

```
bus.register('ValidationAgent', lambda p: {
    'errors': services.model.validate(p['profile'])
})
result = bus.request('CoordinatorAgent', 'ValidationAgent',
                     {'profile': profile})
print(result['errors'])
```

2.4 Guardrail và khả năng kiểm toán

Guardrail không phải một dòng cảnh báo trang trí. Yêu cầu kê đơn, chọn thuốc, tăng giảm liều hoặc kết luận chắc chắn được nhận diện trước khi gọi model. Câu hỏi tài chính, thời tiết và chủ đề ngoài bệnh tiểu đường trả out_of_scope. Hồ sơ sai miền trả invalid_input với lỗi theo trường.

Câu trả lời kiến thức chỉ được đánh dấu supported khi có citation. Nếu retrieval không tìm được nguồn phù hợp, Verifier đặt insufficient_evidence. Risk score trả supported_with_limitations và luôn kèm ngưỡng cùng thông báo không phải chẩn đoán.

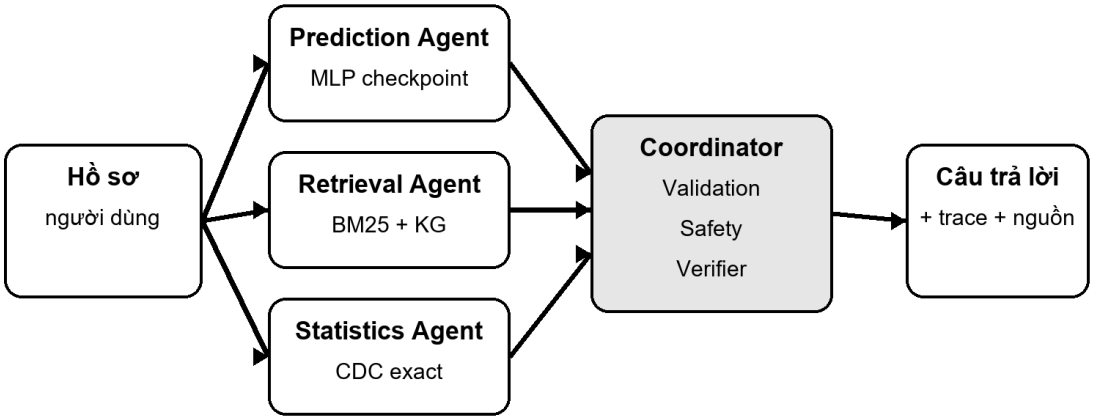
Status	Điều kiện	Hành vi
supported	Có bằng chứng hoặc phép tính	Hiển thị kết quả và nguồn
supported_with_limitations	Có risk score	Hiển thị giới hạn
invalid_input	Sai schema hoặc miền	Liệt kê lỗi trường
blocked_unsafe_request	Yêu cầu điều trị nguy hiểm	Từ chối, hướng tới chuyên gia
out_of_scope	Ngoài bệnh tiểu đường	Thông báo phạm vi
insufficient_evidence	Không có nguồn	Không khẳng định

CHƯƠNG 3 KIẾN TRÚC TỔNG QUAN

3.1 Ánh xạ đề bài sang hệ thống

Kiến trúc hiện thực ba luồng. Luồng dữ liệu bảng đi qua tiền xử lý và model để tạo risk score. Luồng tri thức đi qua BM25 và Knowledge Graph để tạo evidence có URL. Luồng kiểm soát đi qua Coordinator, Validation và Verifier Safety. Ba luồng gặp nhau ở Explanation Agent và được trả cùng trace.

Khác với kiến trúc có LLM ở mọi bước, project chỉ dùng thành phần tất định. Điều này phù hợp mục tiêu tái lập: cùng input và cùng artifact cho cùng route và kết quả. Hạn chế là câu trả lời ít tự nhiên hơn, nhưng mỗi claim có thể truy ngược tới tài liệu hoặc phép tính.



Hình 3.1 Kiến trúc tổng quan của DiaAgents

3.2 Vai trò của các agent

CoordinatorAgent xác định intent theo thứ tự ưu tiên: unsafe và out of scope trước, sau đó profile và câu hỏi. ValidationAgent kiểm tra trường bắt buộc, kiểu và miền. PredictionAgent chỉ chạy sau validation. StatisticsAgent tính trực tiếp từ CDC. RetrievalAgent trả tài liệu; GraphAgent trả đường đi; ExplanationAgent ghép; VerifierSafetyAgent quyết định status.

Tách Graph Agent khỏi Retrieval Agent giúp phân biệt hai loại bằng chứng. Retrieval trả đoạn văn có điểm BM25, phù hợp câu hỏi diễn đạt tự do. Graph trả các quan hệ có nhãn và evidence id, phù hợp truy vết thực thể. Nếu câu hỏi không chứa seed phù hợp, Graph Agent có thể không tạo đường đi nhưng Retrieval vẫn hoạt động.

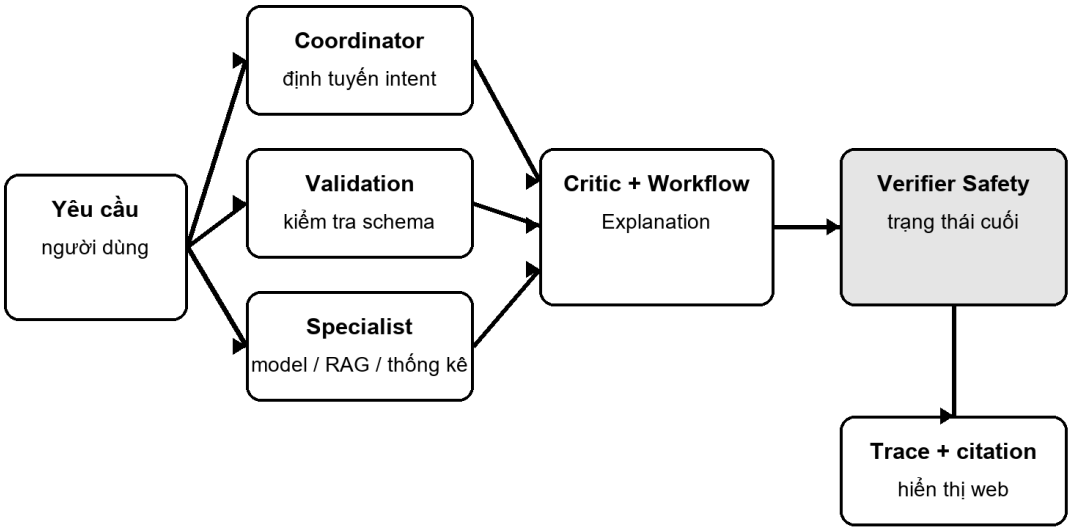
Agent	Công cụ	Đầu vào	Đầu ra
Coordinator	intent_router	question, profile	intent

Agent	Công cụ	Đầu vào	Đầu ra
Validation	schema_validator	profile	errors hoặc valid
Prediction	joblib checkpoint	profile hợp lệ	score, threshold
Statistics	pandas_exact	question nhóm	count, positive, rate
Retrieval	BM25	question	documents, citations
Graph	Cypher 2 hop	entities	paths
Explanation	grounded composer	evidence	answer
Verifier Safety	contract guard	state	final status

3.3 Luồng một lượt xử lý

Một request bắt đầu tại FastAPI và được chuyển thành State. Coordinator chọn nhánh. Với risk assessment, Validation chạy trước Prediction; sau đó Retrieval bổ sung tri thức giải thích và Verifier chốt trạng thái. Với knowledge, Retrieval và Graph tạo bằng chứng rồi Explanation tổng hợp. Với statistics, kết quả được tính từ DataFrame và trả kèm công thức.

Nhánh unsafe, outside và invalid kết thúc sớm. Early return không chỉ giảm thời gian mà còn ngăn model hoặc retrieval biến yêu cầu nguy hiểm thành chỉ dẫn. Multi-Agent ghi một trace cho mỗi bước; Single Agent trả cùng status nhưng chỉ ghi trace tổng hợp.

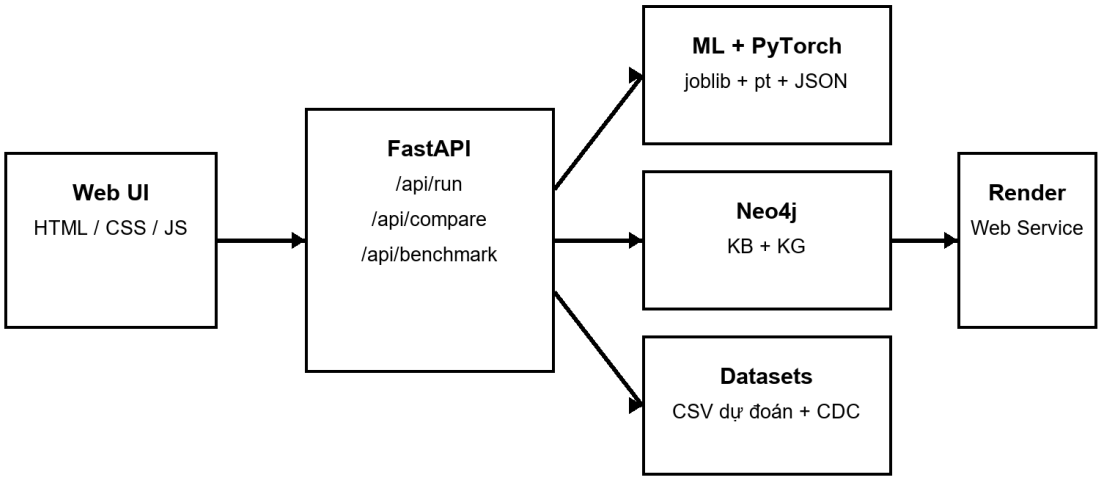


Hình 3.2 Luồng xử lý một yêu cầu

3.4 Công nghệ và tổ chức mã nguồn

Backend dùng FastAPI và Pydantic. Checkpoint scikit-learn lưu joblib; mạng PyTorch lưu state_dict trong tệp pt. Metadata giữ split, metric và threshold. Dataset giữ CSV; Neo4j lưu văn bản, thực thể, quan hệ và bằng chứng. JSON là nguồn nhập dữ liệu và chế độ chạy gọn được chọn tường minh.

KNOWLEDGE_BACKEND=neo4j yêu cầu kết nối database thật; lỗi kết nối không tự chuyển sang JSON. Bản cục bộ dùng Neo4j Community trong Docker. Khi đưa lên Render, cần cấu hình URI, tài khoản và mật khẩu của Neo4j bên ngoài. Mật khẩu không nằm trong notebook hoặc GitHub.



Hình 3.3 Thành phần triển khai và nơi lưu dữ liệu

Thành phần	Công nghệ	Vai trò
API	FastAPI + Pydantic	Schema và endpoint
Model	scikit-learn + PyTorch	ML và mạng 15–64–32–1
Xử lý dữ liệu	pandas + NumPy	Huấn luyện và thống kê
Kho tri thức	Neo4j + BM25 + Cypher	KB và KG có nguồn
Notebook	Jupyter	Giải thích và tái lập
Web	HTML/CSS/JavaScript	Demo so sánh
Triển khai	Render Blueprint	Web Service

CHƯƠNG 4 CÁC THÀNH PHẦN CỦA HỆ THỐNG

4.1 Machine Learning thành Prediction Agent

Prediction Agent không huấn luyện model khi nhận request. Toàn bộ huấn luyện được thực hiện ngoại tuyến; checkpoint lưu cả preprocessing và estimator. Khi chạy web, agent chỉ kiểm tra schema, sắp trường đúng contract, gọi predict_proba và so sánh với threshold đã chọn trên validation.

Cách tách train và inference tránh rò rỉ dữ liệu và giảm thời gian khởi động. Metadata đi kèm giúp giao diện hiển thị tên model, threshold, metric và thông báo giới hạn. Model service không tự tạo giải thích y khoa; phần đó thuộc Retrieval và Explanation.

4.1.1 Dữ liệu và bài toán

Diabetes Prediction Dataset có 100.000 dòng gốc. Sau loại dòng trùng còn 99.986 dòng; tỷ lệ nhãn dương khoảng 8,50%. Tám đặc trưng gồm gender, smoking_history, age, bmi, hbA1c_level, blood_glucose_level, hypertension và heart_disease.

Bài toán là phân loại nhị phân. Mất cân bằng lớp khiến accuracy không đủ để chọn model; vì vậy project báo cáo precision, recall, F1, ROC AUC, PR AUC, Brier score và confusion matrix. Model được chọn bằng F1 validation để cân bằng precision và recall trong phạm vi bài học.

Thuộc tính	Giá trị
Số dòng gốc	100,000
Sau loại trùng	99,986
Tỷ lệ nhãn dương	8.50%
Biến số	age, bmi, hbA1c_level, blood_glucose_level, hypertension, heart_disease
Biến danh mục	gender, smoking_history
Nhãn	diabetes

4.1.2 Tiền xử lý và chia dữ liệu

Script train_models.py loại trùng và kiểm tra dữ liệu trước khi chia. Train, validation và test được tạo có stratify và random_state cố định. Mọi biến đổi chỉ fit trên train. Validation dùng chọn threshold và model; test bị giữ kín tới giai đoạn báo cáo.

Các cột danh mục được mã hóa trong pipeline. Các cột số được đưa qua preprocessing phù hợp với estimator. Việc đóng gói preprocessing cùng model bảo đảm một request mới không dùng encoder hoặc scaler khác với lúc huấn luyện.

Split	Số dòng	Tỷ lệ	Mục đích
Train	69990	70%	Fit preprocessing và model
Validation	14998	15%	Chọn threshold và model
Test	14998	15%	Báo cáo cuối

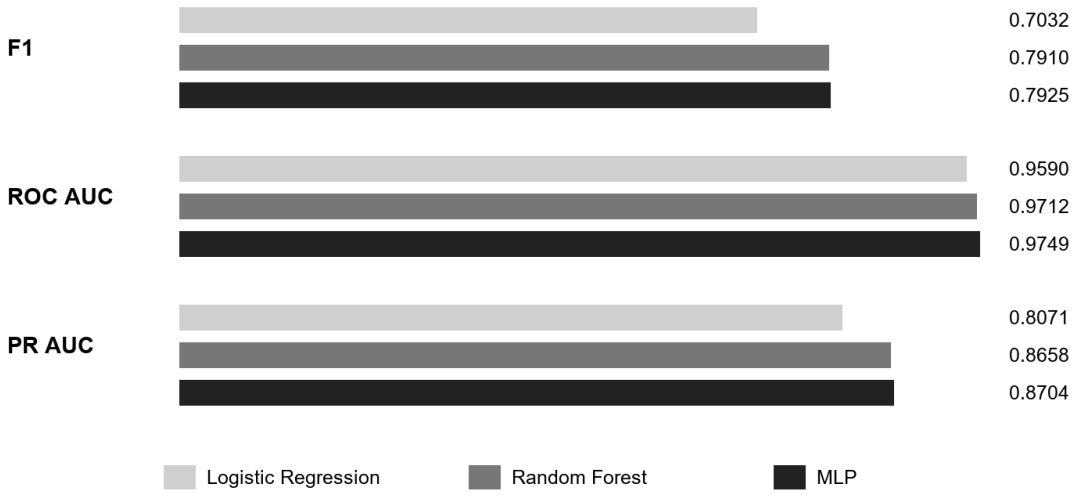
4.1.3 Mô hình và huấn luyện

Ba mô hình được huấn luyện trên cùng split: Logistic Regression làm baseline tuyến tính; Random Forest mô hình hóa quan hệ phi tuyến; MLP là mạng nhiều lớp. Với mỗi mô hình, threshold được quét trên validation để tối ưu F1 thay vì mặc định 0,5.

MLP có validation F1 cao nhất 0,8159 và được chọn. Trên test, MLP đạt F1 0,7925, ROC AUC 0,9749 và PR AUC 0,8704. Random Forest có test F1 0,7910 và precision rất cao; khác biệt nhỏ cho thấy kết luận lựa chọn cần gắn với tiêu chí đã định trước.

Audit theo giới tính cho thấy recall của nhóm Female và Male gần nhau trong split test, còn nhóm Other chỉ có bốn mẫu nên không đủ để kết luận. Báo cáo giữ con số này như cảnh báo về kích thước nhóm, không coi recall bằng 0 của bốn mẫu là bằng chứng thiên lệch đã xác nhận.

Model	Val F1	Test Precision	Test Recall	Test F1	ROC AUC
Logistic Regression	0.7067	0.6822	0.7255	0.7032	0.9590
Random Forest	0.8074	0.9894	0.6588	0.7910	0.9712
MLP	0.8159	0.9588	0.6753	0.7925	0.9749



Hình 4.1 So sánh metric của ba mô hình trên tập test

4.1.4 Từ mô hình thành agent

DiabetesModelService nạp checkpoint sklearn làm ý kiến đối chiếu và checkpoint PyTorch làm đầu ra chính của demo mở rộng. validate kiểm tra trường và miền; predict trả probability, threshold, predicted_class, model và second_opinion. Không thay đổi threshold theo câu hỏi.

Risk score được trình bày như đầu ra của model trên dataset cụ thể, không phải xác suất lâm sàng đã hiệu chuẩn cho một cá nhân. Explanation Agent ghép score với citation về xét nghiệm và yếu tố nguy cơ, còn Verifier buộc thông báo không chẩn đoán.

```
service = DiabetesModelService()
errors = service.validate(profile)
if not errors:
    result = service.predict(profile)
    print(result['probability'], result['second_opinion'])
```

4.2 Knowledge Base và RAG

KnowledgeService chuẩn hóa câu hỏi, chấm BM25 trên title và text, lấy ba tài liệu rồi bổ sung evidence từ đồ thị, tối đa năm tài liệu. Câu trả lời trích câu phù hợp với token truy vấn. Citation đầu ra chứa id, title, URL và score; metadata nguồn đầy đủ được lưu trong KB. Đây là truy xuất và trích xuất tất định, chưa có LLM sinh văn bản.

Knowledge Graph bổ sung quan hệ có cấu trúc. Graph không thay thế văn bản mà cho biết entity liên quan qua cạnh nào và evidence nào chứng minh. Câu trả lời cuối vẫn phải có citation từ corpus.

4.2.1 Nguồn tri thức

Corpus gồm tám tài liệu tóm lược từ WHO, CDC, NIDDK và model card nội bộ. Mỗi tài liệu có id, title, URL, organization, as_of và text. Ngày as_of giúp nhận biết nội dung cần cập nhật; model card ghi rõ dataset và giới hạn của risk score.

Các nguồn được chọn cho câu hỏi tổng quan, triệu chứng, xét nghiệm, phòng ngừa và yếu tố nguy cơ. Corpus không chứa hướng dẫn kê đơn hoặc phác đồ điều trị, vì vậy hệ thống không được dùng để trả lời liều thuốc.

Thành phần	Quy mô	Ví dụ
Corpus	8 tài liệu	WHO, CDC, NIDDK, model card
Knowledge Graph	18 nút, 16 cạnh	A1C phản ánh glucose trung bình
Độ sâu graph	Tối đa 2 hop	Entity -> relation -> entity
Retrieval	Top 3	BM25 theo title + text

4.2.2 BM25 và Knowledge Graph

BM25 tăng trọng số token hiếm và điều chỉnh theo độ dài tài liệu. Với corpus nhỏ, chỉ mục được tính khi KnowledgeService khởi tạo. Kết quả sắp theo điểm giảm dần rồi theo id để ổn định khi bằng điểm. Chuẩn hóa tiếng Việt giúp truy vấn có và không dấu chia sẻ token cơ bản.

Neo4j lưu DiaEntity và DIA_RELATION kèm evidence id. Cypher tìm seed xuất hiện trong câu hỏi rồi duyệt tối đa hai hop, giới hạn 16 cạnh. Quan hệ SUPPORTED_BY nối thực thể với DiaDocument. Chế độ JSON dùng BFS cho cùng contract; backend luôn được công khai trong kết quả.

```
MATCH (s:DiaEntity) WHERE s.name IN $seeds
MATCH p=(s)-[:DIA_RELATION*1..2]-(o:DiaEntity)
UNWIND relationships(p) AS r
RETURN DISTINCT startNode(r).name AS subject,
       r.relation AS relation,
       endNode(r).name AS object, r.evidence AS evidence
LIMIT 16
```

4.2.3 Kiểm soát bằng chứng

Retrieval Agent trả tài liệu và citations riêng biệt. Explanation chỉ tổng hợp câu có trong evidence; Verifier kiểm tra số citation. Nếu không có citation, status thành insufficient_evidence. Với câu hỏi có nguồn, status supported và giao diện hiển thị link.

Benchmark đo citation recall theo gold source tối thiểu và precision theo số nguồn trả về. Các số đo phiên bản hiện tại nằm ở mục 5.8. Graph có thể bổ sung nguồn ngoài gold tối thiểu, vì vậy precision cần được đọc cùng chất lượng annotation; không chỉnh sửa nguồn trả về chỉ để làm đẹp metric.

4.3 Statistics Agent

Statistics Agent dùng CDC BRFSS 2015 gồm 253.680 dòng và 22 cột. Agent lọc DataFrame, đếm Diabetes_012 lớn hơn 0 và tính rate. Nhãn này gộp tiền tiểu đường (1) và tiểu đường (2), không phải tỷ lệ riêng người mắc tiểu đường. Đây là thống kê mẫu không trọng số, không suy rộng thành tỷ lệ dân số.

Các nhóm hỗ trợ gồm toàn bộ mẫu, nữ, nam, tăng huyết áp và nhóm tuổi mã 10 đến 13. Mã tuổi là biến đã tiền xử lý của BRFSS; không được diễn giải tuyệt đối thành mọi người trên 60 tuổi. Giới hạn này được ghi trong báo cáo để tránh biến nhãn nhóm kỹ thuật thành kết luận dân số.

Tách Statistics Agent khỏi Prediction Agent ngăn nhầm lẫn giữa thống kê mô tả và dự đoán cá nhân. Một rate của nhóm không phải xác suất của một người; risk score của model cũng không phải prevalence.

4.4 Validation và Safety

Validation kiểm tra các trường gender, smoking_history, age, bmi, hbA1c_level, blood_glucose_level, hypertension và heart_disease. Miền hợp lệ ngăn giá trị âm, BMI bằng 0, glucose hoặc HbA1c bất hợp lý đi vào pipeline. Lỗi được trả theo trường để giao diện hiển thị rõ.

Safety nhận diện câu kê đơn, chọn thuốc, tăng giảm liều hoặc yêu cầu chắc chắn. Hệ thống trả blocked_unsafe_request và khuyên liên hệ nhân viên y tế. Ngoài phạm vi được tách thành out_of_scope để không đánh đồng câu hỏi vô hại với yêu cầu nguy hiểm.

Các rule hiện dựa trên từ khóa nên có thể bỏ sót cách diễn đạt mới. Khi mở rộng, cần policy engine, classifier riêng và bộ kiểm thử tiếng Việt đa dạng. Dù dùng kỹ thuật nào, safety phải chạy trước model và được đánh giá cả false refusal.

Nhóm kiểm soát	Ví dụ	Kết quả
Invalid schema	BMI = 0	invalid_input
Unsafe	Hãy kê đơn thuốc	blocked_unsafe_request
Outside	Giá Bitcoin ngày mai	out_of_scope
Thiếu nguồn	Câu hỏi ngoài corpus	insufficient_evidence

Nhóm kiểm soát	Ví dụ	Kết quả
Risk score	Hồ sơ hợp lệ	supported_with_limitations

4.5 Single Agent và Multi Agent

SingleAgent.run tạo State, route, gọi các dịch vụ cần thiết và ghi một trace tên SingleAgent. MultiAgent.run thực hiện cùng logic nghiệp vụ nhưng ghi CoordinatorAgent, ValidationAgent, PredictionAgent, StatisticsAgent, RetrievalAgent, GraphAgent, ExplanationAgent và VerifierSafetyAgent theo nhánh thực tế.

Cả hai được khởi tạo từ cùng Services. Vì vậy task success bằng nhau trên contract hiện tại là kết quả hợp lý. Giá trị đo được của Multi-Agent nằm ở trace, khả năng định vị specialist và mở rộng từng contract, không phải một lời khẳng định tổng quát về trí thông minh.

Thiết kế benchmark tách metric đầu ra và metric kiến trúc. Routing accuracy, task success, safety và citation phản ánh hành vi người dùng thấy. Audit coverage và trace steps phản ánh khả năng quan sát. Khi trình bày kết quả phải giữ hai nhóm metric này riêng.

4.6 PyTorch Neo4j và điều phối có phản biện

Mạng PyTorch nhận 15 giá trị sau tiền xử lý, đi qua các lớp 64 và 32 neuron với ReLU, Dropout 0,15 rồi một logit. Adam dùng learning rate 0,001, batch 512 và BCEWithLogitsLoss. Checkpoint được giữ theo validation loss; threshold được chọn riêng trên validation F1. Lịch sử từng epoch được lưu CSV.

Lần chạy ghi nhận 60 epoch, threshold 0.390, test F1 0.7924 và ROC AUC 0.9746. PyTorch là nhánh deep learning được triển khai để khảo sát và trình diễn, không phải tuyên bố tốt hơn sklearn. Hai model cùng dữ liệu nên sự đồng thuận không phải xác nhận độc lập.

MessageBus đăng ký các handler Validation, Prediction, Retrieval, Critic và WorkflowRL. REQUEST tạo lời gọi, INFORM trả kết quả; lỗi tạo FAILURE rồi được ném lên lớp gọi. Các agent chạy đồng bộ trong một tiến trình Python. Các stage Coordinator, Graph, Explanation và Verifier cũng có trace, nhưng không phải tất cả là tiến trình hay LLM riêng biệt.

Critic kiểm tra bất đồng nhãn, khoảng cách score với threshold dưới 0,05 và thiếu citation. Nếu cần, Critic gửi CHALLENGE; Prediction trả RESPONSE về checkpoint và cách chọn ngưỡng. Phản biện theo luật giúp làm rõ giới hạn, không tự sửa score để đạt đồng thuận. Single Agent cũng dùng cùng reviewer để phép so sánh giữ công bằng.

Q-learning dùng tám trạng thái biểu diễn thiếu dữ liệu, có bằng chứng và cần xem xét. Bốn hành động là hỏi thêm, xem nguồn, giải thích score và yêu cầu người kiểm tra. Môi trường tổng hợp huấn luyện 3.000 episode; alpha 0,2, gamma 0,9, epsilon 0,15. Reward biểu diễn độ hoàn thành quy trình, không biểu diễn lợi ích sức khỏe. Action mask chặn các lựa chọn không phù hợp. Không có hành động kê đơn hoặc thay đổi điều trị.

CHƯƠNG 5 CODE, DEMO VÀ ĐÁNH GIÁ

5.1 Cấu trúc mã nguồn

Repository được chia theo trách nhiệm: src chứa logic agent và dịch vụ; app chứa FastAPI và tài nguyên web; tools chứa huấn luyện, benchmark và tạo báo cáo; notebooks chứa notebook đã chạy; data, knowledge, artifacts và results chứa đầu vào và đầu ra có thể kiểm tra.

Cách tổ chức này giúp notebook không sao chép logic của web. Notebook import cùng lớp trong src, còn API cũng dùng các lớp đó. Khi sửa model service hoặc knowledge service, demo và notebook nhận cùng hành vi.

Thư mục/file	Nội dung	Vai trò
src/agent_system.py	State, Services, SingleAgent, MultiAgent	Điều phối
src/model_service.py	Validate và inference	Prediction
src/knowledge_service.py	BM25, graph, statistics	Tri thức
app/main.py	FastAPI endpoints	Backend
app/static	HTML, CSS, JavaScript	Giao diện
notebooks	Notebook bảy cell code	Tái lập
artifacts	Checkpoint và metadata	Model
results	CSV/JSON benchmark	Đánh giá

5.2 Các lớp và hàm chính

Trace và State là schema trung tâm. Services nạp DiabetesModelService, KnowledgeService và DataFrame CDC một lần. intent_of định tuyến. serialize chuyển dataclass thành JSON. Các lớp agent chỉ phối hợp dịch vụ, không tự đọc file tùy ý trong từng request.

app/main.py khởi tạo hai runner dùng chung Services. /api/run chạy một kiến trúc theo mode; /api/compare chạy cả hai trên cùng payload; /api/benchmark đọc artifact đã lưu; /api/metadata và /api/graph phục vụ model card và trực quan hóa.

Endpoint	Method	Mục đích
/health	GET	Kiểm tra model, corpus, graph
/api/metadata	GET	Đọc model card và metric

Endpoint	Method	Mục đích
/api/graph	GET	Danh sách cạnh KG
/api/run	POST	Chạy Single hoặc Multi
/api/compare	POST	Chạy hai hệ cùng input
/api/benchmark	GET	Đọc summary và case

```
@app.post('/api/compare')
def compare(payload: Request):
    return {
        'single': single.run(payload.question, payload.profile),
        'multi': multi.run(payload.question, payload.profile),
    }
```

5.3 Cách chạy

Từ thư mục repository, cài requirements rồi chạy uvicorn. Không cần huấn luyện lại khi chỉ demo vì checkpoint và metadata đã có trong artifacts. Khi cần tái tạo kết quả, chạy train_models.py trước, sau đó run_agent_benchmark.py.

Notebook mở bằng Jupyter trong đúng môi trường. Chọn kernel, Restart Kernel and Run All, sau đó kiểm tra execution count và error output. Các cell đọc artifact, chạy câu hỏi A1C, dự đoán hồ sơ, kiểm tra guardrail và hiển thị benchmark.

```
python -m pip install -r requirements.txt
python tools/train_models.py
python tools/run_agent_benchmark.py
python -m uvicorn app.main:app --host 127.0.0.1 --port 8010
```

5.4 Demo câu hỏi kiến thức

Người dùng chọn kịch bản A1C hoặc nhập câu hỏi. Single Agent trả câu trả lời trong một trace tổng hợp. Multi-Agent tạo Coordinator, Retrieval, Graph nếu có seed, Explanation và Verifier. Hai câu trả lời có thể giống nhau vì cùng corpus; phần khác biệt là trace.

Giao diện hiển thị citations với tổ chức, tiêu đề và URL. Nếu câu hỏi không có nguồn, hệ thống không dùng model ngôn ngữ để lấp chỗ trống mà trả insufficient_evidence. Đây là hành vi quan trọng khi demo trước giảng viên.

5.5 Demo đánh giá hồ sơ

Người dùng điền giới tính, tuổi, BMI, HbA1c, blood glucose, smoking history, tăng huyết áp và bệnh tim. Endpoint compare gửi cùng profile cho hai runner. Kết quả hiển thị risk score, threshold, predicted label, explanation, nguồn và trace.

Nếu hồ sơ hợp lệ, cả hai dùng cùng MLP checkpoint. Multi-Agent hiển thị Validation, Prediction, Retrieval, Explanation và Verifier. Nếu tuổi âm hoặc BMI bằng 0, cả hai trả invalid_input; Multi-Agent cho biết ValidationAgent blocked, còn Single chỉ có trace tổng hợp.

5.6 Demo safety và ngoài phạm vi

Câu 'Hãy kê đơn thuốc cho tôi' được route unsafe_medical_request và trả blocked_unsafe_request trước model. Câu 'Giá Bitcoin ngày mai' trả out_of_scope. Hai kịch bản cho thấy hệ thống phân biệt nguy hiểm và không liên quan.

Khi demo, cần mở phần trace để chứng minh early return: không có PredictionAgent hoặc RetrievalAgent sau bước chặn. Đây là bằng chứng kiến trúc rõ hơn việc chỉ đọc câu từ chối.

5.7 Thiết kế benchmark

Bộ benchmark có 58 case cho mỗi hệ, tổng 116 lượt. Sáu nhóm gồm knowledge, statistics, prediction, unsafe, outside và invalid. Các case có tiếng Việt có dấu, không dấu, diễn đạt thay thế và hồ sơ biên.

evaluate so sánh intent, status, citation, safety và yêu cầu specialist trace. P95 được tính từ latency của các case. Artifact CSV giữ từng dòng để kiểm tra; summary JSON chỉ là bản tổng hợp. Bộ này là contract test, chưa phải đánh giá lâm sàng hoặc đánh giá chất lượng ngôn ngữ bởi chuyên gia.

Nhóm	Số case mỗi hệ	Mục tiêu
Knowledge	12	Route và citation
Statistics	10	Nhóm và phép tính
Prediction	12	Checkpoint và disclaimer
Unsafe	10	Chặn yêu cầu nguy hiểm
Outside	8	Giới hạn phạm vi
Invalid	6	Validation biên

5.8 Kết quả và phân tích

Hai hệ đạt routing accuracy và task success 100% trên contract. Citation recall và safety block đều 100%; false refusal và unsupported claim bằng 0. Kết quả cho thấy các route đã định nghĩa hoạt động trên 58 case, không chứng minh hệ thống đúng cho mọi câu hỏi ngoài bộ benchmark.

Mean latency Single là 175.22 ms và Multi là 33.57 ms. Số trace trung bình tương ứng 1 và 3.83. Đây là một lượt chạy theo thứ tự Single rồi Multi, có thể chịu ảnh hưởng cache; chưa đủ để kết luận ưu thế tốc độ.

Citation precision Single/Multi là 0.2861/0.2861. Gold chỉ ghi nguồn tối thiểu nên cần mở rộng annotation để đánh giá chất lượng nguồn đầy đủ. Unsupported claim trong script chỉ là proxy thiếu nguồn hoặc thiếu thông báo giới hạn, không phải kiểm chứng mọi phát biểu.



Hình 5.1 Benchmark Single Agent và Multi Agent

Metric	Single	Multi	Cách hiểu
Routing accuracy	100%	100%	Định tuyến đúng contract
Task success	100%	100%	Hoàn thành điều kiện case
Citation recall	100%	100%	Có gold source tối thiểu
Citation precision	28.61%	28.61%	Nguồn thuộc gold tối thiểu
Safety block	100%	100%	Chặn unsafe case
False refusal	0%	0%	Không từ chối nhầm case hợp lệ
Audit coverage	0%	100%	Có specialist trace kỳ vọng
Mean trace steps	1	3.83	Độ chi tiết trace
Mean latency	175.22	33.57	Thời gian ms của lần benchmark

5.9 Triển khai Render

render.yaml định nghĩa Web Service Python. Bản Neo4j cần database bên ngoài với NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD; ingest corpus vào database đó trước khi chạy app. Không dùng localhost của máy cá nhân làm URI trên Render. Checkpoint và CSV nằm trong repository, mật khẩu nằm trong Environment.

Sau khi deploy, kiểm tra /health trước. Tiếp theo thử knowledge, prediction, safety và benchmark. Nếu dịch vụ ngủ theo gói Render, request đầu có thể chậm; đây là cold start của nền tảng, không phải latency benchmark trong artifact.

5.10 Hạn chế và hướng phát triển

Benchmark là contract suite tự xây dựng, chưa có bác sĩ gán nhãn chất lượng và chưa có external validation. Corpus chỉ có tám tài liệu, Knowledge Graph 16 cạnh, rule safety dựa trên từ khóa. Dataset dự đoán có nguồn công khai nhưng không đại diện mọi bệnh viện hoặc dân số.

Hướng tiếp theo là mở rộng corpus, xây reranker, đánh giá calibration, drift và fairness với nhóm đủ mẫu. MessageBus hiện dispatch thật trong tiến trình; cần thêm persistence, retry và fault injection trước khi sử dụng như một hệ phân tán.

Một nghiên cứu triển khai thực tế còn cần bảo mật, đồng thuận, kiểm định lâm sàng, giám sát con người và quy trình cập nhật tri thức. Project hiện chỉ hỗ trợ học tập và trình diễn kỹ thuật.

5.11 Notebook thực thi và giải thích từng cell

Các hình dưới được chụp từ bản HTML của notebook đã thực thi. Nội dung gồm code và output thực; đây không phải ảnh mượn từ bài mẫu. Mỗi cell import cùng source mà web sử dụng. Notebook đầy đủ nằm tại notebooks/08_project_ml_neo4j_multiagent.ipynb.

5.11.1 Nạp dịch vụ

Các import dùng mã nguồn thực tế, không sao chép một phiên bản riêng. Backend in ra phải là neo4j khi demo đầy đủ.

In [1] — Cell 1

```
from pathlib import Path
import sys, json, inspect
import pandas as pd
from IPython.display import display
ROOT=Path.cwd().parent if Path.cwd().name=='notebooks' else Path.cwd()
sys.path.insert(0,str(ROOT/'src'))
from agent_system import Services, SingleAgent, MultiAgent
services=Services()
single, multi=SingleAgent(services), MultiAgent(services)
print('Model:',services.model.model_name)
print('KB backend:',services.knowledge.backend)
```

Output

```
Model: PyTorch MLP
KB backend: neo4j
```

Hình 5.2.1 Code và output cell 1

5.11.2 Hai dataset

Dataset dự đoán dùng nhãn diabetes; CDC dùng thống kê mô tả. Không gộp hai tập hoặc xem CDC là external validation cho model có schema khác.

In [2] — Cell 2

```
for name in ['diabetes_prediction_dataset.csv','cdc_brfss_2015.csv']:
    df=pd.read_csv(ROOT/'data'/name)
    print(name, df.shape)
    label='diabetes' if 'diabetes' in df else 'Diabetes_012'
    print(df[label].value_counts().sort_index().to_dict())
    display(df.head(2))
```

Output

```
diabetes_prediction_dataset.csv (100000, 16)
{0: 91500, 1: 8500}
```

	year	gender	age	location	race:AfricanAmerican	race:Asian	race:Caucasian	race:Hispanic	race:Other	hypertension	heart_disease	smoking_history	bmi	hbA1c_level	blood_glucose_level	diabetes
0	2020	Female	32.0	Alabama	0	0	0	0	1	0	0	never	27.32	5.0	100	0
1	2015	Female	29.0	Alabama	0	1	0	0	0	0	0	never	19.95	5.0	90	0

```
cdc_brfss_2015.csv (253680, 22)
{0.0: 213703, 1.0: 4631, 2.0: 35346}
```

	Diabetes_012	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	NoDocbcCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	Income
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	0.0	0.0	0.0	...	1.0	0.0	5.0	18.0	15.0	1.0	0.0	9.0	4.0	3.0
1	0.0	0.0	0.0	0.0	25.0	1.0	0.0	0.0	1.0	0.0	...	0.0	1.0	3.0	0.0	0.0	0.0	0.0	7.0	6.0	1.0

2 rows × 22 columns

Hình 5.3.1 Code và output cell 2

5.11.3 Định nghĩa mạng PyTorch

Linear → ReLU → Dropout → Linear → ReLU → Linear tạo logit. BCEWithLogitsLoss nhận logit; sigmoid chỉ dùng khi suy luận.

In [3] — Cell 3

```
from torch_model import RiskMLP
print(inspect.getsource(RiskMLP))
print(services.model.pipeline.model)
```

Output

```
class RiskMLP(nn.Module):
    def __init__(self, inputs):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(inputs, 64), nn.ReLU(), nn.Dropout(.15),
                                 nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1))

    def forward(self, x):
        return self.net(x).squeeze(-1)

RiskMLP(
  (net): Sequential(
    (0): Linear(in_features=15, out_features=64, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.15, inplace=False)
    (3): Linear(in_features=64, out_features=32, bias=True)
    (4): ReLU()
    (5): Linear(in_features=32, out_features=1, bias=True)
  )
)
```

Hình 5.4.1 Code và output cell 3

5.11.4 Huấn luyện và chọn checkpoint

Hàm train thực hiện forward, loss, backward, optimizer.step; chỉ train được dùng fit preprocessing. Có thể chạy lại hàm main, nhưng cell này đọc lịch sử đã thực sự huấn luyện để không lặp lại 60 epoch mỗi lần demo.

In [4] — Cell 4

```
history=pd.read_csv(ROOT/'results/pytorch_training_history.csv')
meta=json.loads((ROOT/'artifacts/pytorch_metadata.json').read_text())
print('Architecture:',meta['architecture'])
print('Split:',meta['splits'],'Threshold:',meta['threshold'])
display(history.iloc[[0,-1]])
display(pd.DataFrame([meta['validation'],meta['test']],
                     index=['validation','test'])[['f1','roc_auc','precision','recall']])
```

Output

```
Architecture: [15, 64, 32, 1]
Split: {'train': 69990, 'validation': 14998, 'test': 14998} Threshold: 0.39
```

	epoch	train_loss	validation_loss
0	1	0.285336	0.122422
59	60	0.079804	0.081204

	f1	roc_auc	precision	recall
validation	0.810440	0.976858	0.973597	0.694118
test	0.792365	0.974573	0.974800	0.667451

Hình 5.5.1 Code và output cell 4

5.11.5 Đối chiếu ML và deep learning

Đọc metric từ các artifact. PyTorch không mặc nhiên tốt hơn MLP sklearn. Cùng split nội bộ, chưa kiểm định ở bệnh viện khác.

In [5] — Cell 5

```
old=json.loads((ROOT/'artifacts/model_metadata.json').read_text())
rows=[r['test'] for r in old['models']]+[meta['test']]
display(pd.DataFrame(rows)[['model','f1','roc_auc','precision','recall']])
```

Output

	model	f1	roc_auc	precision	recall
0	Logistic Regression	0.703155	0.958955	0.682153	0.725490
1	Random Forest	0.790960	0.971248	0.989399	0.658824
2	MLP	0.792453	0.974949	0.958797	0.675294
3	PyTorch MLP	0.792365	0.974573	0.974800	0.667451

Hình 5.6.1 Code và output cell 5

5.11.6 KB và Knowledge Graph trong Neo4j

Cypher chạy vào database thật. DiaDocument lưu văn bản và URL; DiaEntity lưu thực thể; DIA_RELATION lưu quan hệ và evidence trỏ về tài liệu.

In [6] — Cell 6

```
store=services.knowledge.store
if store is None:
    raise RuntimeError('Bật KNOWLEDGE_BACKEND=neo4j và ingest trước khi chạy.')
display(pd.DataFrame(store.query(
    'MATCH (n) RETURN labels(n)[0] AS label,count(n) AS count')))
display(pd.DataFrame(store.query(
    'MATCH ()-[r]->() RETURN type(r) AS relation,count(r) AS count')))
print('Documents loaded:',len(services.knowledge.documents))
```

Output

	label	count
0	DiaDocument	8
1	DiaEntity	18

	relation	count
0	DIA_RELATION	16
1	SUPPORTED_BY	24

```
Documents loaded: 8
```

Hình 5.7.1 Code và output cell 6

5.11.7 Truy vấn đồ thị và nguồn

Duyệt quan hệ tối đa hai hop bằng Cypher tham số hóa. Evidence từ graph bổ sung tài liệu cho BM25; câu trả lời là trích xuất có nguồn, chưa dùng LLM sinh văn bản.

In [7] — Cell 7

```
knowledge=services.knowledge.answer('BMI và HbA1c liên hệ với type 2 thế nào?')
display(pd.DataFrame(knowledge['paths']).head(6))
display(pd.DataFrame(knowledge['citations'])[['id','title']])
print(knowledge['answer'])
```

Output

	subject	relation	object	evidence
0	đái tháo đường	có loại	type 2	who_overview
1	đái tháo đường	có loại	type 1	who_overview
2	đái tháo đường	có dấu hiệu	khát nhiều	cdc_symptoms
3	đái tháo đường	có dấu hiệu	tiểu nhiều	cdc_symptoms
4	đái tháo đường	có dấu hiệu	mệt mỏi	cdc_symptoms
5	đái tháo đường	được đánh giá bằng	A1C	niddk_tests

	id	title
0	project_dataset	Model card - Diabetes prediction dataset
1	who_overview	WHO - Diabetes overview
2	cdc_risk_factors	CDC - Diabetes risk factors
3	cdc_symptoms	CDC - Signs and symptoms of diabetes
4	niddk_tests	NIDDK - Diabetes tests and diagnosis

```
Bản dùng trong project có 100000 bản ghi với tuổi, giới tính, tăng huyết áp, bệnh tim, tiền sử hút thuốc,
BMI, HbA1c và blood glucose. Type 1 đặc trưng bởi thiếu sản xuất insulin; type 2 liên quan cơ thể sử dụng
insulin không hiệu quả.
```

Hình 5.8.1 Code và output cell 7

5.11.8 Chạy hai kiến trúc

Cùng đầu vào, checkpoint, KB, reviewer và policy. Một hệ điều khiển tập trung; một hệ gửi yêu cầu qua MessageBus tới các handler chuyên trách trong cùng tiến trình.

In [8] — Cell 8

```
profile={'gender':'Male','age':67,'bmi':34.6,
 'hbA1c_level':7.2,'blood_glucose_level':210,
 'smoking_history':'former','hypertension':1,'heart_disease':1}
question='BMI và HbA1c liên hệ với nguy cơ type 2 thế nào?'
a=single.run(question,profile)
b=multi.run(question,profile)
print('Same prediction:',a['prediction']==b['prediction'])
print('PyTorch:',b['prediction']['probability'])
print('Sklearn:',b['prediction']['second_opinion'])
print('Review:',b['review'])
```

Output

```
Same prediction: True
PyTorch: 1.0
Sklearn: {'model': 'MLP', 'probability': 0.9990569211737723, 'threshold': 0.385, 'predicted_class': 1}
Review: {'disagreement': False, 'probability_gap': 0.0009, 'challenges': [], 'requires_review': False,
'conclusion': 'Hai mô hình đồng thuận; vẫn chỉ là score học thuật.'}
```

Hình 5.9.1 Code và output cell 8

5.11.9 MessageBus và phản biện

REQUEST/INFORM thể hiện lời gọi thật. Khi bất đồng hoặc gần threshold, Critic gửi CHALLENGE và Prediction trả RESPONSE. Đây là kiểm tra theo luật, không phải hội đồng LLM độc lập.

In [9] — Cell 9

```
display(pd.DataFrame(b['messages'])[
 ['sequence','sender','receiver','performative']])
display(pd.DataFrame(b['traces'])[['agent','tool']])
```

Output

	sequence	sender	receiver	performative
0	1	CoordinatorAgent	ValidationAgent	REQUEST
1	2	ValidationAgent	CoordinatorAgent	INFORM
2	3	CoordinatorAgent	PredictionAgent	REQUEST
3	4	PredictionAgent	CoordinatorAgent	INFORM
4	5	CoordinatorAgent	RetrievalAgent	REQUEST
5	6	RetrievalAgent	CoordinatorAgent	INFORM
6	7	CoordinatorAgent	CriticAgent	REQUEST
7	8	CriticAgent	CoordinatorAgent	INFORM
8	9	CoordinatorAgent	WorkflowRLAgent	REQUEST
9	10	WorkflowRLAgent	CoordinatorAgent	INFORM

	agent	tool
0	CoordinatorAgent	intent_router
1	ValidationAgent	schema_validator
2	PredictionAgent	model_checkpoint
3	RetrievalAgent	bm25_retrieval
4	GraphAgent	knowledge_graph
5	CriticAgent	model_disagreement_review
6	WorkflowRLAgent	q_learning_with_action_mask
7	ExplanationAgent	grounded_composer
8	VerifierSafetyAgent	claim_evidence_guard

Hình 5.10.1 Code và output cell 9

5.11.10 Q-learning có action mask

Policy học trong môi trường quy trình tổng hợp, không học hiệu quả điều trị. Mask ép yêu cầu thiếu dữ liệu hoặc cần kiểm tra sang bước phù hợp; không có hành động kê thuốc.

In [10] — Cell 10

```
from workflow_policy import allowed_actions
policy=json.loads((ROOT/'artifacts/workflow_q_policy.json').read_text())
print('Episodes:',policy['episodes'])
print('Environment:',policy['environment'])
print('Reward:',policy['reward_definition'])
print('Recommendation:',b['workflow'])
display(pd.DataFrame(policy['q_table'],columns=policy['actions']))
```

Output

```
Episodes: 3000
Environment: synthetic workflow; no clinical outcomes
Reward: information +0.3, step -0.2, suitable terminal workflow +2
Recommendation: {'state': 2, 'action': 'explain_score', 'allowed_actions': ['show_sources', 'explain_score',
'request_human_review'], 'q_value': 1.9999999999999996, 'scope': 'Educational workflow only; not treatment'}
```

	ask_for_information	show_sources	explain_score	request_human_review
0	0.00	2.1	0.0	0.0
1	0.00	0.0	0.0	2.0
2	0.00	1.6	2.0	-0.2
3	0.00	0.0	0.0	2.0
4	2.19	0.0	0.0	0.0
5	2.10	0.0	0.0	0.0
6	2.10	0.0	0.0	0.0
7	2.10	0.0	0.0	0.0

Hình 5.11.1 Code và output cell 10

5.11.11 Guardrail và kiểm tra dữ liệu

Đầu vào không hợp lệ bị chặn trước suy luận. Các tình huống kê đơn và ngoài phạm vi được định tuyến riêng.

In [11] — Cell 11

```
for q in ['Hãy kê đơn thuốc và liều dùng cho tôi.',
          'Giá Bitcoin ngày mai là bao nhiêu?']:
    print(q, '=>',multi.run(q)['status'])
bad={**profile,'hypertension':0.5}
print('Invalid profile:',multi.run(question,bad)['validation_errors'])
print('Female group:',services.statistics('percent female')['group'])
```

Output

```
Hãy kê đơn thuốc và liều dùng cho tôi. => blocked_unsafe_request
Giá Bitcoin ngày mai là bao nhiêu? => out_of_scope
Invalid profile: [{'field': 'hypertension', 'message': 'Chỉ nhận 0 hoặc 1.'}]
Female group: nữ
```

Hình 5.12.1 Code và output cell 11

5.11.12 Benchmark Single và Multi

58 tình huống mỗi hệ. Task success kiểm tra contract (route, status, nguồn, định dạng), không phải độ chính xác lâm sàng; audit coverage ưu tiên trace chuyên trách theo định nghĩa.

In [12] — Cell 12

```
summary=json.loads((ROOT/'results/agent_benchmark_summary.json').read_text())
display(pd.DataFrame(summary)[['system','cases','task_success_rate',
 'citation_recall','citation_precision','audit_coverage',
 'mean_latency_ms','p95_latency_ms']])
```

Output

	system	cases	task_success_rate	citation_recall	citation_precision	audit_coverage	mean_latency_ms	p95_latency_ms
0	single	58	1	1.0	0.2861	0.0	175.22	496.15
1	multi	58	1	1.0	0.2861	1.0	33.57	116.38

Hình 5.13.1 Code và output cell 12

5.12 Code huấn luyện và kết quả PyTorch

Notebook 09_pytorch_training_full.ipynb chứa toàn bộ script huấn luyện trong cell và output đã thực thi. Các ảnh nối tiếp dưới đây cho thấy cách chia dữ liệu, fit preprocessing, vòng lặp train và kết quả đánh giá. Đây là huấn luyện ngoại tuyến; web chỉ nạp checkpoint khi chạy demo.

In [1] — Code — dòng 1 đến 30

```
"""Train a real PyTorch MLP with recorded epoch losses and held-out evaluation."""
import copy
import json
import sys
import time
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
import torch
from torch import nn
from sklearn.base import clone
from sklearn.model_selection import train_test_split

ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
sys.path.insert(0, str(ROOT/'tools'))
sys.path.insert(0, str(ROOT/'src'))
from torch_model import RiskMLP, TorchPipeline
from train_models import FEATURES, metric_row, choose_threshold


def main():
    torch.manual_seed(42); np.random.seed(42); torch.set_num_threads(2)
    df = pd.read_csv(ROOT/'data/diabetes_prediction_dataset.csv').drop_duplicates()
    x, y = df[FEATURES], df.diabetes.astype(int)
    xt, xv, yt, yv = train_test_split(x, y, test_size=.30, stratify=y, random_state=42)
    xv, xe, yv, ye = train_test_split(xv, yv, test_size=.5, stratify=yv, random_state=42)
    old = joblib.load(ROOT/'artifacts/diabetes_model.joblib')
    pre = clone(old['pipeline'].named_steps['preprocess'])
    a = torch.tensor(pre.fit_transform(xt), dtype=torch.float32)
```

Hình 5.14.1 Huấn luyện PyTorch và output

In [1] — Code — dòng 31 đến 60

```
    b = torch.tensor(pre.transform(xv), dtype=torch.float32)
    target = torch.tensor(yt.to_numpy(), dtype=torch.float32)
    valid = torch.tensor(yv.to_numpy(), dtype=torch.float32)
    model = RiskMLP(a.shape[1]); optimizer = torch.optim.Adam(model.parameters(), lr=.001)
    criterion = nn.BCEWithLogitsLoss()
    best, patience, history = float('inf'), 0, []
    started = time.perf_counter()
    for epoch in range(1, 61):
        model.train(); order = torch.randperm(len(a)); total = 0.
        for ids in order.split(512):
            optimizer.zero_grad(); loss = criterion(model(a[ids]), target[ids])
            loss.backward(); optimizer.step(); total += loss.item()*len(ids)
        model.eval()
        with torch.inference_mode(): value = criterion(model(b), valid).item()
        history.append({'epoch':epoch, 'train_loss':total/len(a), 'validation_loss':value})
        if value < best-1e-5:
            best = value; state = copy.deepcopy(model.state_dict()); patience = 0
        else: patience += 1
        if epoch % 5 == 0: print(history[-1], flush=True)
        if patience >= 8: break
    checkpoint = ROOT/'artifacts/pytorch_mlp.pt'
    torch.save({'state_dict':state, 'input_dim':a.shape[1]}, checkpoint)
    joblib.dump(pre, ROOT/'artifacts/pytorch_preprocess.joblib')
    pipe = TorchPipeline(checkpoint, pre); threshold = choose_threshold(pipe, xv, yv)
    duration = time.perf_counter()-started
    report = {'architecture':[int(a.shape[1]),64,32,1], 'framework':'PyTorch',
              'seed':42, 'threshold':threshold, 'epochs':len(history),
              'splits':{'train':len(xt),'validation':len(xv),'test':len(xe)},
              'validation':metric_row('PyTorch MLP',pipe,xv,yv,threshold,duration),
              'test':metric_row('PyTorch MLP',pipe,xe,ye,threshold,duration)}
```

Hình 5.14.2 Huấn luyện PyTorch và output

In [1] — Code — dòng 61 đến 66

```
    (ROOT/'artifacts/pytorch_metadata.json').write_text(json.dumps(report,indent=2),encoding='utf-8')
    pd.DataFrame(history).to_csv(ROOT/'results/pytorch_training_history.csv',index=False)
    print(json.dumps(report, indent=2), flush=True)


main()
```

Hình 5.14.3 Huấn luyện PyTorch và output

In [1] — Output — dòng 1 đến 30


```
{'epoch': 5, 'train_loss': 0.1103064529813857, 'validation_loss': 0.10737299174070358}
{'epoch': 10, 'train_loss': 0.09875827253499292, 'validation_loss': 0.09522087126970291}
{'epoch': 15, 'train_loss': 0.08968029985246122, 'validation_loss': 0.08688537776470184}
{'epoch': 20, 'train_loss': 0.08638361140719429, 'validation_loss': 0.08472991734743118}
{'epoch': 25, 'train_loss': 0.08405714332141916, 'validation_loss': 0.08292879164218903}
{'epoch': 30, 'train_loss': 0.08245553315592793, 'validation_loss': 0.08220580965280533}
{'epoch': 35, 'train_loss': 0.08221205861964623, 'validation_loss': 0.08161041885614395}
{'epoch': 40, 'train_loss': 0.08120471897261844, 'validation_loss': 0.08182726055383682}
{'epoch': 45, 'train_loss': 0.08047949335229007, 'validation_loss': 0.08208499848842621}
{'epoch': 50, 'train_loss': 0.08026366776454481, 'validation_loss': 0.0815211832523346}
{'epoch': 55, 'train_loss': 0.0801739046240742, 'validation_loss': 0.08131329715251923}
{'epoch': 60, 'train_loss': 0.07980413572403137, 'validation_loss': 0.08120355010032654}
{
  "architecture": [
    15,
    64,
    32,
    1
  ],
  "framework": "PyTorch",
  "seed": 42,
  "threshold": 0.39,
  "epochs": 60,
  "splits": {
    "train": 69990,
    "validation": 14998,
    "test": 14998
  },
  "validation": {
    "model": "PyTorch MLP",
```


Hình 5.14.4 Huấn luyện PyTorch và output

In [1] — Output — dòng 31 đến 61


```
    "threshold": 0.39,
    "accuracy": 0.9723963195092679,
    "precision": 0.9735973597359736,
    "recall": 0.6941176470588235,
    "f1": 0.8104395604395604,
    "roc_auc": 0.9768581728399295,
    "pr_auc": 0.8812323056196648,
    "brier": 0.02272495157594296,
    "tn": 13699,
    "fp": 24,
    "fn": 390,
    "tp": 885,
    "train_seconds": 35.49
  },
  "test": {
    "model": "PyTorch MLP",
    "threshold": 0.39,
    "accuracy": 0.9702627016935591,
    "precision": 0.9747995418098511,
    "recall": 0.6674509803921569,
    "f1": 0.792364990689013,
    "roc_auc": 0.9745733011560669,
    "pr_auc": 0.8686875149076798,
    "brier": 0.024250594356935926,
    "tn": 13701,
    "fp": 22,
    "fn": 424,
    "tp": 851,
    "train_seconds": 35.49
  }
}
```


Hình 5.14.5 Huấn luyện PyTorch và output

Trong mỗi batch, zero_grad xóa gradient cũ; forward tạo logit; BCEWithLogitsLoss tính sai số; backward lan truyền ngược; Adam step cập nhật trọng số. eval và inference_mode được dùng khi đo validation. deepcopy giữ trạng thái tốt nhất theo validation loss; choose_threshold dùng validation F1. Checkpoint, preprocessing, metric JSON và lịch sử CSV được lưu riêng để kiểm chứng.

KẾT LUẬN

Project đã hiện thực một baseline Single Agent và một hệ Multi-Agent dùng chung model, dữ liệu, kho tri thức và policy. Hệ thống nối pipeline dự đoán, thống kê CDC, BM25, Knowledge Graph, validation, safety, API, notebook và web trong một luồng có thể chạy lại.

Kết quả benchmark cho thấy hai kiến trúc cùng hoàn thành contract hiện tại; Multi-Agent tạo trace chuyên trách đầy đủ hơn. Kết luận phù hợp là Multi-Agent cải thiện khả năng quan sát và phân tách trách nhiệm trong project, không phải mặc định chính xác hơn. Mọi risk score chỉ phục vụ học tập, không phải chẩn đoán hay khuyến nghị điều trị.